The Journal of Open Source Software

DeepVariant-ARM64: INT8-quantized variant calling on Linux AArch64

Antonio Lamb [1]

1 Lamb Consulting, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

DeepVariant (Poplin et al., 2018) is the most accurate open-source variant caller for germline short-read sequencing, but it has no official Linux ARM64 build. This fork provides the first open-source ARM64 port of DeepVariant v1.9.0, including a post-training INT8-quantized ONNX model that is 74% smaller (21 MB vs. 84 MB) and 2.3× faster at inference than the FP32 baseline — while preserving WGS accuracy (SNP F1 $\Delta<0.0001$ vs. same-platform FP32 on chr20). A parallel `call_variants` wrapper breaks the GEMM saturation ceiling at 16+ threads, yielding an additional 1.7–2.5× CV speedup. Together, these optimizations enable full 30× WGS variant calling at $0.80/genome on Oracle A1 (dedicated ARM64 vCPUs) or $0.33/genome on Hetzner CAX41 (shared vCPU), compared to ~$5/genome on the standard x86 configuration.

Statement of need

ARM64 server adoption is accelerating in genomics, driven by platforms such as AWS Graviton, Oracle Ampere, and Hetzner Cloud ARM. Yet DeepVariant's build system assumes x86 (SSE4/AVX) throughout, and an upstream request for ARM64 support (Issue #834) has remained unresolved since 2024. The only existing ARM64 option is Sentieon DNAscope (Freed et al., 2017) — a proprietary tool requiring a commercial license, which limits access for academic and resource-constrained laboratories.

This fork addresses the gap by patching the Bazel build system, fixing eight upstream compilation errors for ARM64/GCC 13, and shipping a validated Docker image. The enabling technical contribution is INT8 static quantization of DeepVariant's InceptionV3 model using ONNX Runtime with calibration data drawn from real genomic TFRecords. Post-training INT8 quantization typically degrades accuracy by 0.6–3% on vision CNNs (Nagel et al., 2021); that it does not here — not even in difficult genomic regions such as segmental duplications and homopolymer runs — is the central finding. The explanation is that InceptionV3's large, dense convolutions are unusually quantization-friendly, lacking the depthwise separable layers that cause INT8 degradation in architectures like EfficientNet and MobileNet (Jacob et al., 2018).

An important engineering detail: INT8 quantization can produce probability vectors that do not sum to 1.0. Without correction, `postprocess_variants` crashes with `ValueError: Invalid genotype likelihoods`. The ONNX inference path in this fork renormalizes outputs before passing them to `round_gls()`. This fix is mandatory for INT8 deployment and is included in the Docker image.

DRAFT

Key results

Accuracy was validated with `rtg vcfeval` (Cleary et al., 2015) against the Genome in a Bottle HG003 v4.2.1 truth set (Zook et al., 2019).

INT8 quantization preserves WGS accuracy on the same ARM64 platform:

Metric	FP32 (chr20)	BF16 (chr20)	INT8 ONNX (chr20)	INT8 Full Genome
SNP F1	0.9977	0.9977	0.9978	0.9961
INDEL F1	0.9961	0.9961	0.9962	0.9956

The chr20-to-full-genome drop is expected (harder chromosomes contribute more errors). The gap vs. x86 (~0.9996 SNP F1) likely reflects differences in the ARM64 build environment (TensorFlow version, OneDNN backend, numerical precision paths) rather than INT8 quantization, since the chr20 INT8 result matches or exceeds FP32 and BF16 on the same ARM64 platform.

For WES (HG003 IDT exome 100x), INT8 matches the FP32 baseline (SNP F1 0.9931 vs. 0.9930, Overall F1 0.9923 vs. 0.9924), with a small INDEL F1 reduction (-0.0038) within expected post-training quantization variance.

Five-platform benchmarking corpus:

Platform	Best config	$/genome	Wall time (30x WGS est.)
AWS Graviton4 (c8g)	INT8 + 4-way CV	$3.13	~2.4 hr
AWS Graviton3 (c7g)	BF16 + jemalloc	$3.43	~5.9 hr
Oracle A2 (AmpereOne)	INT8 + 4-way CV	$2.14	~4.3 hr
Oracle A1 (Altra)	INT8 + 4-way CV	$0.80	~5.0 hr
Hetzner CAX41 (Altra)	INT8 + jemalloc	$0.33	~7.7 hr

Documented dead ends

An EfficientNet-B3 model swap was built end-to-end but ran 3× slower than InceptionV3 on ARM64 CPUs despite 3.2× fewer FLOPs. Depthwise separable convolutions and squeeze-and-excitation blocks have poor GEMM density on CPU NEON hardware, negating their theoretical FLOP advantage. Full details are provided in `TRAINING_EXPERIMENT.md`.

Software availability and reproducibility

The software is BSD-3 licensed and available at https://github.com/antomicblitz/deepvariant-linux-arm64. A Docker image is published on GitHub Container Registry. Nextflow and Snakemake workflows are provided in `workflows/`. Reproduction scripts (`scripts/benchmark_full_chr20.sh`, `scripts/validate_accuracy.sh`) allow independent verification of all reported accuracy and performance numbers on any ARM64 Linux machine with Docker installed.

AI usage disclosure

Claude Opus 4 (Anthropic, 2025–2026) was used for code generation assistance, build system debugging, documentation drafting, and test scaffolding throughout this project. All AI-assisted

DRAFT

outputs were reviewed, validated, and modified by the author. Core design decisions — including the INT8 renormalization fix for `postprocess_variants`, platform-specific backend selection logic (OneDNN/SIGILL avoidance on AmpereOne), the parallel `call_variants` architecture, and the GEMM saturation analysis — were made by the human author. Benchmark data was collected and validated independently by the author on physical cloud instances across five ARM64 platforms. The AI tool was not used to generate or fabricate any experimental results.

Acknowledgements

The author thanks Google for releasing DeepVariant as open-source software, the Genome in a Bottle Consortium for the HG003 truth set, and the ONNX Runtime team for ARM64 INT8 quantization support.

References

Cleary, J. G., Braithwaite, R., Gaastra, K., Hilbush, B. S., Inglis, S., Irvine, S. A., Jackson, A., Littin, R., Rathod, M., Ware, D., Zook, J. M., Hallet, L., & Knight, D. (2015). Comparing variant call files for performance benchmarking of next-generation sequencing variant calling pipelines. *bioRxiv*. https://doi.org/10.1101/023754

Freed, D., Aldana, R., Weber, J. A., & Edwards, J. S. (2017). The Sentieon genomics tools – a fast and accurate solution to variant calling from next-generation sequence data. *bioRxiv*. https://doi.org/10.1101/115717

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713. https://doi.org/10.1109/CVPR.2018.00286

Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., Baalen, M. van, & Blankevoort, T. (2021). A white paper on neural network quantization. *arXiv Preprint arXiv:2106.08295*. https://doi.org/10.48550/arXiv.2106.08295

Poplin, R., Chang, P.-C., Alexander, D., Schwartz, S., Colthurst, T., Ku, A., Newburger, D., Dijamco, J., Nguyen, N., Afshar, P. T., Gross, S. S., Dorfman, L., McLean, C. Y., & DePristo, M. A. (2018). A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology*, *36*(10), 983–987. https://doi.org/10.1038/nbt.4235

Zook, J. M., McDaniel, J., Olson, N. D., Wagner, J., Parber, H., Antez, F., Lambert, C., Garnett, C., Sawyer, W. M., Paul, L., Greer, B., King, K., Green, E. D., Xiao, C., Sherry, S., & Salit, M. (2019). An open resource for accurately benchmarking small variant and reference calls. *Nature Biotechnology*, *37*(5), 561–566. https://doi.org/10.1038/s41587-019-0074-6

DRAFT